MINI-PROJET POO · SUJET 3

# Portefeuille d'investissement : Mizen Patrimoine

*Cahier des charges pédagogique*

**Wissem Zeddini** · À lire avec le document « Mini-projets POO : cadre commun »

| | |
|---|---|
| **Client** | Mme Sonia Khemiri, directrice du cabinet Mizen Patrimoine (client simulé) |
| **Le logiciel** | Suivi et évaluation d'un portefeuille d'actifs très différents : actions, cryptomonnaies, biens immobiliers et comptes d'épargne |
| **Difficulté** | Intermédiaire (2 sur 3), avec des calculs en entiers à soigner |
| **Notions du socle** | Encapsulation, constructeurs, héritage, redéfinition, polymorphisme |
| **Bonus possibles** | Abstraction (badge Abstrait), exception personnalisée (badge Résilient), sauvegarde dans un fichier (badge Archiviste) |
| **Livrable final** | Code, fichier .jar, UML, rapport de 2 à 3 pages, démonstration de 5 minutes |

## 1. Le contexte

Le cabinet **Mizen Patrimoine** (un cabinet fictif) conseille des familles qui possèdent des biens très différents : des actions, des cryptomonnaies, un appartement loué, un livret d'épargne. Aujourd'hui, ses conseillers tiennent tout cela dans des tableurs séparés, et personne ne sait répondre vite à deux questions : « Combien vaut le portefeuille ? » et « Que devient-il si le marché chute ? ».

Le cabinet a besoin d'un logiciel qui suit chaque actif, évalue le portefeuille d'un seul coup et estime les impôts. **Vous êtes l'équipe de développement.** La cliente est exigeante et elle changera d'avis : concevez dès le départ pour que votre code puisse évoluer.

*Les taux d'imposition de ce sujet sont **inventés pour l'exercice** : ils ne correspondent pas à la fiscalité réelle.*

## 2. Le vocabulaire du métier

| Terme | Sens pour le client |
|---|---|
| **Actif** | Un bien que le cabinet suit pour un client : une action, une cryptomonnaie, un bien immobilier ou un compte d'épargne. |
| **Solde** | L'argent liquide rattaché à un actif (liquidités, loyers reçus, intérêts), en millimes. |
| **Dépôt, retrait** | Les deux seules façons de faire varier un solde ; chacune est contrôlée. |
| **Historique** | Le journal des dépôts et des retraits d'un actif, que personne ne doit pouvoir falsifier. |
| **Valeur actuelle** | Ce que vaut un actif aujourd'hui : sa valeur de marché, plus son solde. |
| **Portefeuille** | L'ensemble des actifs d'un client, avec une capacité maximale. |
| **Valeur nette** | La somme des valeurs actuelles de tous les actifs du portefeuille. |
| **Plus-value** | Le gain d'un actif dont la valeur de marché dépasse la somme investie ; elle n'est jamais négative pour l'impôt. |

| Terme | Sens pour le client |
|---|---|
| **Variation de marché** | Un pourcentage entier (par exemple +5 ou -12) appliqué au marché pendant un tour. |
| **Tour, scénario** | Un tour est une période de simulation (un trimestre). Un scénario est une suite de variations, une par tour, fournie en données : aucun hasard. |
| **Revenu** | Les loyers ou les intérêts versés dans le solde de l'actif à chaque tour. |
| **Impôt** | La somme due sur un actif selon son régime fiscal, en millimes (1 dinar = 1 000 millimes). |

# 3. Les règles du métier

Ces règles sont celles de la cliente. Chacune doit être **protégée par votre code**, pas seulement respectée par vos tests.

| N° | Règle | Apparaît en |
|---|---|---|
| **R1** | Un solde ne devient **jamais négatif** : un retrait supérieur au solde est refusé. | v0.2 |
| **R2** | Un dépôt ou un retrait porte sur un **montant strictement positif**. | v0.2 |
| **R3** | Chaque opération **réussie** est notée dans l'historique ; une opération refusée n'y figure jamais ; l'extérieur peut **lire** l'historique, jamais le modifier. | v0.2 |
| **R4** | Chaque type d'actif calcule **sa propre valeur actuelle** (tableau ci-dessous). | v0.3 |
| **R5** | Dans un portefeuille, un **identifiant est unique**, et la capacité maximale ne peut pas être dépassée. | v0.3 |
| **R6** | La variation de marché est un **pourcentage entier par tour** ; chaque type y réagit à sa façon, et une valeur de marché **ne descend jamais sous 0** (une variation de -100 % ou moins la ramène à 0). | v1.0 |
| **R7** | Les **revenus** (loyers, intérêts) sont versés dans le solde à chaque tour. | v1.0 |
| **R8** | Chaque type a **son régime fiscal** ; un impôt est un entier en millimes, **jamais négatif**. | v1.0 |

### Les types d'actifs

| Type | Valeur actuelle | Réaction au marché | Revenu par tour | Impôt (taux fictifs) |
|---|---|---|---|---|
| **Actif générique (classe mère)** | Son solde | Aucune | Aucun | Aucun |
| **Action** | Prix unitaire × quantité, plus le solde | Le prix varie du pourcentage du tour | Aucun | 15 % de la plus-value positive |
| **Cryptomonnaie** | Valeur de marché, plus le solde | Variation × 3 (forte volatilité) | Aucun | 30 % de la plus-value positive |
| **Bien immobilier** | Valeur du bien, plus le solde | Variation ÷ 4 (division entière) | Un loyer fixe versé dans le solde | 20 % des loyers perçus |
| **Compte d'épargne** | Son solde | Aucune | Intérêts : 1 % du solde (taux fixe) | 10 % des intérêts perçus au-delà de 20 000 millimes |

*Utilisez des **constantes nommées** plutôt que des nombres écrits au milieu du code. L'argent est un **entier en millimes** : prenez le type `Long`, car un `int` s'arrête vers 2,1 milliards et le produit « valeur × pourcentage » le dépasse vite.*

# 4. Version v0.1 : Cadrage

**Mail du client · v0.1**

**De :** Sonia Khemiri, cabinet Mizen Patrimoine

**Objet :** Notre futur outil de suivi

Bonjour,

Mon cabinet gère des portefeuilles très variés, et mes conseillers y perdent un temps fou. Avant de coder quoi que ce soit, montrez-moi que vous avez compris mon métier : les objets que vous avez repérés, ce que chacun connaît de lui-même, et un premier actif affiché à l'écran. Pas d'écran graphique : la console me suffit.

Cordialement.

| N° | Demande du client | Notion |
|---|---|---|
| **1.1** | « En tant que gestionnaire de patrimoine, je veux que vous identifiiez les objets de mon métier, afin que nous parlions le même langage. » | Analyse, UML |
| **1.2** | « Je veux voir un actif à l'écran, afin de vérifier que mon besoin est compris. » | Classe, constructeur, main |

### Vous livrez

- Un **UML de départ** d'au moins 3 classes (par exemple ActifFinancier, Portefeuille et Demo), avec leurs attributs principaux.
- Des **squelettes de classes qui compilent**.
- Une classe **Demo** dont le main affiche un actif, par exemple : `L001 · Compte courant · valeur 500000 millimes`.

**Indice**

Un actif connaît au moins son **identifiant**, son **nom** et son **solde**. Ne cherchez pas encore les quatre types : ils arrivent avec le mail de la version v0.3.

# 5. Version v0.2 : Objets fiables

**Mail du client · v0.2**

**De :** Sonia Khemiri

**Objet :** Des soldes négatifs et un historique falsifiable

Bonjour,

Merci pour votre première version. Mes conseillers ont fait des essais et j'ai une mauvaise surprise : un retrait trop gros a laissé un **solde négatif**, et un dépôt de -100 millimes a été accepté ! Pire : un stagiaire a pu **effacer une ligne de l'historique** d'un compte depuis un autre écran. Pour un cabinet financier, c'est inacceptable : un historique falsifiable ne vaut rien.

Mes règles sont simples : un solde ne descend jamais sous zéro, on ne dépose et on ne retire que des montants positifs, chaque opération réussie est notée dans l'historique, et personne ne doit pouvoir modifier le solde autrement que par un dépôt ou un retrait contrôlé.

Cordialement.

| N° | Demande du client | Notion |
|---|---|---|
| **2.1** | « En tant que responsable de la conformité, je veux que le solde ne change que par des dépôts et des retraits vérifiés, afin d'éviter les erreurs et les fraudes. » (R2) | Encapsulation, validation |
| **2.2** | « Je veux qu'un retrait supérieur au solde soit refusé, afin qu'aucun solde ne soit négatif. » (R1) | Méthode métier |
| **2.3** | « Je veux que chaque dépôt ou retrait réussi soit noté dans l'historique, et jamais un refus, afin de garder une trace fidèle. » (R3) | Tableau privé |
| **2.4** | « Je veux lire l'historique sans pouvoir le modifier, afin de l'auditer. » (R3) | Encapsulation d'un tableau |
| **2.5** | « Je veux créer un actif avec ou sans solde de départ, afin de gagner du temps. » | Constructeurs surchargés |
| **2.6** | « Je veux lire un actif en une ligne, afin de contrôler vite. » | toString() |

## Vous livrez

- Tous les attributs **privés**, et **aucun setter pour le solde ni pour l'historique** : seuls le dépôt et le retrait modifient le solde.
- Deux constructeurs (avec et sans solde de départ) qui **valident** le solde reçu.
- Une **Demo** avec au moins trois actifs, qui montre **des refus** (retrait trop gros, montant négatif ou nul) et l'historique d'un actif.

> **Indices**
>
> Testez les **valeurs limites** : un retrait égal au solde (accepté), supérieur d'un millime (refusé), un montant de 0 ou de -100. Un piège classique : un getter qui renvoie **directement** le tableau de l'historique laisse l'extérieur le modifier. Renvoyez une **copie**. Pensez à valider aussi dans le constructeur.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Solde initial invalide (-5000) : 0 utilisé.
L001 · Compte courant · valeur 500000 millimes
L002 · Compte travaux · valeur 0 millimes
L003 · Compte test · valeur 0 millimes
--- Dépôts et retraits ---
Retrait refusé : solde insuffisant (solde 550000, demandé 900000).
Dépôt refusé : le montant doit être positif (reçu : -100 millimes).
Retrait refusé : le montant doit être positif (reçu : 0 millimes).
L001 · Compte courant · valeur 550000 millimes
--- Historique de L001 ---
  1. Dépôt +500000 millimes (solde initial)
  2. Dépôt +250000 millimes (virement client)
  3. Retrait -200000 millimes
Après modification de la copie : Dépôt +500000 millimes (solde initial)
Opérations enregistrées : 3
```

# 6. Version v0.3 : La famille

**Mail du client · v0.3**

**De :** Sonia Khemiri

**Objet :** Quatre sortes d'actifs, et ce n'est pas fini

Bonjour,

Excellent travail : plus de solde négatif, et mon historique est enfin fiable. Passons à mes vrais clients ! Ils possèdent quatre sortes d'actifs : des actions, des cryptomonnaies, des biens immobiliers et des comptes d'épargne. Chacun se valorise à sa manière (voir le tableau).

Je veux regrouper les actifs dans un portefeuille, tous types mélangés, les afficher d'un coup et connaître la valeur nette de l'ensemble.

Et une chose importante : bientôt, je proposerai aussi de l'or, des obligations et d'autres produits. **Je ne veux pas que vous réécriviez le portefeuille à chaque nouveau produit.**

Cordialement.

| N° | Demande du client | Notion |
|---|---|---|
| **3.1** | « Je veux quatre types d'actifs avec leurs particularités (quantité, volatilité, loyer, taux), afin de décrire fidèlement mes clients. » | Héritage, super(...) |
| **3.2** | « Je veux que chaque type calcule sa propre valeur actuelle, afin d'appliquer ma façon de valoriser. » (R4) | Redéfinition, @Override |
| **3.3** | « Je veux un portefeuille qui contient des actifs de tous types, avec une capacité maximale et sans doublon d'identifiant, afin de refléter mes dossiers. » (R5) | Tableau ou liste du type commun |
| **3.4** | « Je veux afficher tout le portefeuille en une fois, afin de le contrôler. » | Une seule boucle, toString() |
| **3.5** | « Je veux la valeur nette de l'ensemble, afin de répondre à mes clients. » | Polymorphisme |
| **3.6** | « Je veux ajouter un nouveau type plus tard sans modifier le portefeuille, afin que mon logiciel évolue. » | Preuve du polymorphisme |

### Vous livrez

- Une **classe mère concrète** qui porte ce qui est commun (solde, historique, dépôts, retraits), et **quatre classes filles** qui n'ajoutent que leurs particularités. L'UML est mis à jour.
- Un **Portefeuille** qui contient des actifs de types différents, et **une seule boucle** pour les afficher et calculer la valeur nette.
- La preuve que le portefeuille n'a pas besoin d'être modifié pour accueillir un nouveau type.

**Indices**

Si vous écrivez **instanceof** dans la boucle du portefeuille, c'est que le polymorphisme manque : quelle méthode chaque type devrait-il redéfinir ? La valeur d'une action, c'est ce que la mère sait déjà (le solde) **plus** ce qui lui est propre : appelez `super.calculerValeurActuelle()` plutôt que de recopier le solde. Pour comparer deux identifiants (du texte), utilisez `equals`, jamais `==`.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Identifiant A101 déjà présent : ajout refusé.
Portefeuille plein (5/5) : ajout refusé.
Portefeuille Cèdre (5/5)
  Action A101 · Atlas Telecom · valeur 1250000 millimes · 100 titres à 12500 millimes
  Crypto C201 · ByteCoin · valeur 2000000 millimes · volatilité ×3
  Bien I301 · Studio Jasmins · valeur 180000000 millimes · loyer 900000 millimes
  Épargne E401 · Livret Sérénité · valeur 3000000 millimes · taux 1 % par tour
  L001 · Compte courant · valeur 400000 millimes
Valeur nette : 186650000 millimes
```

# 7. Version v1.0 : Livraison

**Mail du client · v1.0**

**De :** Sonia Khemiri

**Objet :** Dernière étape avant le lancement

Bonjour,

Le portefeuille est clair, et mes conseillers l'adorent. Mes clients me posent maintenant la question qui fâche : « Et si le marché chutait ? ». Je veux **simuler des scénarios** : une suite de variations en pourcentage, une par tour (un tour est un trimestre), que je fournis moi-même, pour que deux simulations identiques donnent exactement le même résultat. Chaque actif réagit à sa façon : la cryptomonnaie amplifie, l'immobilier amortit, l'épargne ignore le marché.

Et la deuxième question : **combien d'impôts ?** Mon ancien logiciel contenait un gros bloc de « si c'est une action… sinon si c'est un bien… » qu'il fallait rouvrir à chaque nouveau produit. Je ne veux plus de ça.

Un menu console pour mes conseillers, et un fichier que je puisse lancer moi-même.

Cordialement.

| N° | Demande du client | Notion |
|---|---|---|
| **4.1** | « Je veux faire passer un tour de marché sur tout le portefeuille, revenus compris, afin de voir l'effet d'une variation. » (R7) | Méthode polymorphe |
| **4.2** | « Je veux jouer un scénario fourni en données (un tableau de pourcentages), afin d'obtenir des simulations reproductibles. » | Tableau d'entiers, boucle |
| **4.3** | « Je veux que chaque type réagisse au marché à sa façon, sans jamais tomber sous 0, afin d'être réaliste. » (R6) | Redéfinition, calcul en entiers |
| **4.4** | « Je veux l'impôt de chaque actif puis l'impôt global, sans que le portefeuille demande le type, afin que mes nouveaux produits s'intègrent seuls. » (R8) | Défi métier, polymorphisme |
| **4.5** | « Je veux un menu console, afin que mes conseillers utilisent le logiciel sans toucher au code. » | Menu, boucle de saisie |
| **4.6** | « Je veux un fichier que je lance moi-même, afin de l'installer sur mon poste. » | Fichier .jar |

**Le défi métier : le choc de marché et la note fiscale**

Pour chaque **tour** du scénario, le portefeuille applique la variation à **tous** ses actifs :

**1.** chaque actif réagit à sa façon (tel quel, ×3, ÷4, ou pas du tout) et sa valeur de marché ne descend jamais sous 0 ;

**2.** les revenus (loyers, intérêts) sont versés dans le solde ;

**3.** on affiche la valeur nette après chaque tour ;

**4.** à la fin, chaque actif calcule **son propre impôt** et le portefeuille les additionne : impôt global, puis valeur nette après impôt.

Le portefeuille **ne demande jamais** « de quel type es-tu ? » : c'est l'actif qui connaît son régime fiscal.

Menu console attendu (exemple) :

```
=== MIZEN PATRIMOINE ===
1. Afficher le portefeuille
2. Déposer sur un actif
3. Retirer d'un actif
4. Valeur nette et impôt global
5. Simuler un scénario de marché
6. Voir l'historique d'un actif
0. Quitter
Votre choix (0 à 6) :
```

Exemple de sortie d'une Demo de simulation, avec le scénario `{8, -12, 5, -10, 20, 10}` (la vôtre peut différer) :

```
Portefeuille Cèdre (4/5)
  Action A101 · Atlas Telecom · valeur 1250000 millimes · 100 titres à 12500 millimes
  Crypto C201 · ByteCoin · valeur 2000000 millimes · volatilité ×3
  Bien I301 · Studio Jasmins · valeur 180000000 millimes · loyer 900000 millimes
  Épargne E401 · Livret Sérénité · valeur 3000000 millimes · taux 1 % par tour
Valeur nette de départ : 186250000 millimes
Scénario de 6 tours.
Tour 1 · marché +8 % · valeur nette 191360000 millimes
Tour 2 · marché -12 % · valeur nette 185727500 millimes
Tour 3 · marché +5 % · valeur nette 188736503 millimes
Tour 4 · marché -10 % · valeur nette 185397569 millimes
Tour 5 · marché +20 % · valeur nette 196133677 millimes
Tour 6 · marché +10 % · valeur nette 201514984 millimes
Bilan fiscal
  A101 · Atlas Telecom : 34770 millimes
  C201 · ByteCoin : 197281 millimes
  I301 · Studio Jasmins : 1080000 millimes
  E401 · Livret Sérénité : 16456 millimes
Impôt global : 1328507 millimes
Valeur nette après impôt : 200186477 millimes
--- Choc extrême : -40 % en un tour ---
Portefeuille Choc (1/1)
  Crypto C201 · ByteCoin · valeur 0 millimes · volatilité ×3
```

**Indices**

La **tentation** : un grand `if` avec **instanceof**, ou un cast comme `(ActionBoursiere) actif`, pour choisir le régime fiscal. C'est exactement le bloc que la cliente refuse : chaque nouveau produit obligerait à rouvrir le portefeuille. La bonne voie : **une seule méthode** `calculerImpot()` que chaque type redéfinit.

Calculez en entiers : valeur × (100 + variation) / 100, jamais valeur × 1,08. Pour l'immobilier, la variation divisée par 4 se fait en division entière (en Java, -10 / 4 donne -2). Un impôt n'est jamais négatif : pensez à `Math.max`.

# 8. Les bonus du sujet

| Badge | Défi |
|---|---|
| **Abstrait** | Rendez la classe mère **abstraite** et faites de `calculerImpot` une méthode abstraite : chaque type est obligé de déclarer son régime fiscal. Que répond Java si vous écrivez `new ActifFinancier(...)` ? Que devient le « compte de liquidités » générique ? Justifiez votre choix dans le rapport (une interface comme `Imposable` est une autre option : comparez). |
| **Résilient** | Créez une **exception personnalisée** (par exemple `SoldeInsuffisantException`) levée par un retrait trop gros, et interceptée par le menu qui affiche un message clair. |
| **Archiviste** | Sauvegardez le portefeuille dans un fichier texte ou CSV à la fermeture (une ligne par actif : type, identifiant, nom, solde et particularités) et rechargez-le au démarrage. Décidez et justifiez ce que vous faites de l'historique. |

# 9. Les pièges fréquents de ce sujet

- **Un instanceof ou un cast pour choisir le régime fiscal** : le polymorphisme est contourné ; un nouveau produit obligerait à modifier le portefeuille. C'est le piège principal de ce sujet.
- **Un solde négatif** : oublier de comparer le retrait au solde, ou ne pas valider dans le constructeur.
- **Un getter qui renvoie le tableau de l'historique** : l'extérieur peut alors le modifier sans passer par votre classe. Renvoyez une copie.
- **Noter une opération refusée** dans l'historique, ou oublier de noter un loyer ou des intérêts versés.
- **Utiliser int pour l'argent** : le produit valeur × pourcentage dépasse vite 2,1 milliards et donne un résultat faux, sans message d'erreur. Utilisez `long`.
- **Calculer en nombres à virgule** (× 1,08 ou × 0,15) : les erreurs d'arrondi s'accumulent ; restez en entiers.
- **Une valeur négative** après une chute : une variation de -40 % sur une cryptomonnaie vaut -120 % avec le coefficient 3 ; plafonnez à 0.
- **Un impôt négatif** sur une moins-value : l'impôt vaut 0, ce n'est pas un crédit.
- **Comparer des identifiants avec ==**, ou **oublier super(...)** dans le constructeur d'une fille.

# 10. Pour votre rapport

En plus de la structure du cadre commun, répondez à ces questions propres au sujet :

- Pourquoi `getHistorique()` renvoie-t-il une copie ? Quelle règle serait violée sinon, et comment pourrait-on le prouver ?
- Quelle est la ligne de code où le **type réel** d'un actif décide de sa valeur ou de son impôt ?
- Quel risque présente un grand bloc de tests de types pour choisir le régime fiscal ? Que faudrait-il modifier pour ajouter de l'or, et combien de classes existantes ?
- Pourquoi `long` plutôt que `int` ? Donnez un calcul de votre projet qui déborderait avec un `int`.

# 11. Modifications en direct possibles à la soutenance

Pendant la démonstration, on pourra vous demander l'une de ces modifications, en quelques minutes :

- Ajouter un **métal précieux** (il réagit à la moitié de la variation, impôt de 12 % de la plus-value) et le voir dans la valeur nette et l'impôt global **sans toucher au portefeuille**.

- Passer le coefficient de volatilité de la cryptomonnaie de 3 à 4.
- Passer la franchise d'impôt de l'épargne de 20 000 à 50 000 millimes.
- Refuser un retrait qui laisserait moins de 10 000 millimes sur un compte d'épargne.
- Afficher les trois dernières opérations de l'historique d'un actif.

# Annexe : les liens entre les classes (indicatif)

Voici seulement les **relations** attendues à l'arrivée. Les attributs et les méthodes, c'est à vous de les concevoir.

| Relation | Sens |
|---|---|
| **ActionBoursiere, CryptoMonnaie, BienImmobilier, CompteEpargne → ActifFinancier** | Héritage : chaque type **est un** actif financier. |
| **Portefeuille → ActifFinancier (de 0 à plusieurs)** | Le portefeuille **contient** des actifs de tous types, par le type commun. |
| **Demo, Menu → Portefeuille** | Les classes de test et de menu **utilisent** le portefeuille. |